

Бөмбөгний машин

Машина IOI-ийн оролцогчдыг зугаацуулахын тулд бөмбөгний машин зохион бүтээжээ. Уг машины дотоод бүтэц нь доорх байдалтай байна:

- Машин нь 0-ээс $N - 1$ хүртлэх тоонуудаар дугаарласан N ширхэг **оройноос** тогтоно. $N - 1$ дугаартай оройг **үндэс** гэж нэрлэнэ.
- u -р ($0 \leq u < N - 1$) орой бүрийн хувьд u -р оройн **эцэг** орой гэж $P[u] > u$ байх $P[u]$ оройг хэлэх ба u -р орой нь $P[u]$ -р оройн **хүү** орой болно. Үндэс нь эцэг оройгүй байна. Хүүгүй оройг **навч** гэж нэрлэнэ.

Та N болон ямар ч u -р оройн эцэг орой $P[u]$ -г **мэдэхгүй** байгаа. Үүний оронд Машина танд навчнуудын тоо болох M тоог болон навчнуудын индексүүд болох $0, 1, \dots, M - 1$ тоонуудыг хэлж өгнө. Таны даалгавар бол машиныг ажиллуулж, түүний дотоод бүтцийг тодорхойлох явдал юм.

Машины орой бүр дээд тал нь нэг **бөмбөг** хадгалж чадах ба бөмбөг сөрөг биш бүхэл тоон **утга** агуулж чадна. Хэрэв ямар нэг орой x утгатай бөмбөгийг хадгалж байгаа бол уг оройн утга x байна гэж хэлнэ. Хэрэв орой бөмбөг хадгалж байгаа бол **бөмбөгтэй** гэх ба эсрэг тохиолдолд **хоосон** байна гэж хэлнэ. Бүх орой анх хоосон байна.

Та хоёр төрлийн үйлдлийг хийж чадна:

- insert(U, X):** X утгатай шинэ бөмбөгийг U -р навч руу оруулахыг оролдох.
 - Хэрэв U дугаартай навч бөмбөгтэй бол уг үйлдэл `false` утгыг буцааж, бөмбөгийг оруулахгүй.
 - Хэрэв U -р навч хоосон бол бөмбөгийг U -р орой дээр байрлуулна. Үүний дараа бөмбөг өөрийн байрлаж буй оройн эцэг орой хоосон байвал тийшээгээ шилжсээр байна. Бөмбөг үндсэнд хүрэх эсвэл эцэг орой нь бөмбөгтэй байх оройнд хүрснээр шилжилт нь зогсоно. Үйлдэл `true` утгыг буцаана.
- collect():** Хэрэв үндэс хоосон бол (машин хоосон бол), `collect` хоосон массив буцаана. Эсрэг тохиолдолд доор үзүүлсэн `traverse` рекурсив процедур машинд байгаа бүх бөмбөгний утгуудыг S массивт цуглуулна. Анх S массив хоосон байх ба `traverse(N - 1)` гэсэн дуудалт хийгдэнэ.

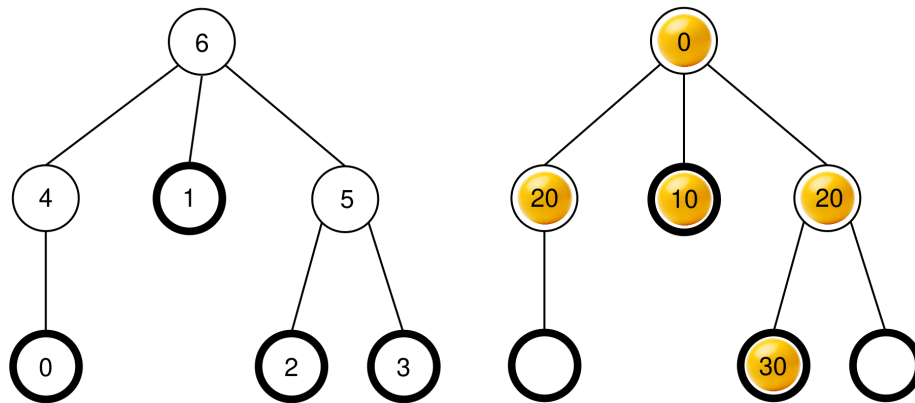
```

traverse(u) :
  u оройн дээрх бөмбөгний утгыг S-ийн төгсгөлд нэмнэ
  c[u] нь u-гийн бөмбөгтэй хүүхдүүдийн жагсаалт байг
  c[u]-г эдгээр оройн дээрх утгуудынх нь үл буурах дарааллаар эрэмбэлэх
    (хэрэв хэд хэдэн орой ижил утгатай бол тэдгээрийг
    ямар ч дарааллаар байрлуулж болно)
  c[u] дахь v бүрийн хувьд :
    traverse(v)

```

Уг процедур ажиллаж дууссаны дараа машинаас **бүх бөмбөгийг гаргах** ба **collect** процедур S-ийг буцаана.

Жишээ нь $N = 7$ ширхэг оройтой, $P = [4, 6, 5, 5, 6, 6]$ эцгийн массив бүхий машиныг авч үзье. Зүүн талын зураг дээр хоосон машиныг оройн индексүүдийн хамт дүрсэлсэн бол баруун талын зураг дээр **insert** үйлдлүүдийн дарааллыг гүйцэтгэсний дараах бөмбөгүүдийн байрлалуудыг дүрсэлсэн (үйлдлүүдийн дараалал Жишээ хэсэгт байгаа).



collect() процедурыг дуудах үед үндэс (6 дугаар орой) бөмбөгтэй байгаа тул S-д анхны утгыг $S = []$ гэж олгоод **traverse(6)** дуудалтыг хийнэ.

traverse(6): 6-р оройн утга 0; түүнийг S рүү нэмснээр $S = [0]$ болно. 6-р оройн хүү оройнууд нь 4, 1 болон 5-р орой ба бүгд бөмбөгтэй байна. Тэдгээрийн утга нь харгалзан 20, 10 ба 20 байна. Үл буурах дарааллаар эрэмбэлснээр $c[6] = [1, 5, 4]$ болно (4 ба 5-р оройнууд ижил утгатай тул $c[6] = [1, 4, 5]$ нь мөн зөв юм). Уг процедур үүний дараа c[6] дахь орой бүр дээр энэ дарааллаар нь **traverse** дуудалтыг хийнэ.

- **traverse(1)**: 1 дүгээр оройн дээр 10 утга байгаа; түүнийг S рүү нэмснээр $S = [0, 10]$ болно. 1-р оройнд бөмбөгтэй хүүхэд байхгүй тул энэ дуудалт дуусна.
- **traverse(5)**: 5-р оройн утга 20; түүнийг S рүү нэмснээр $S = [0, 10, 20]$ болно. 5-р орой нэг бөмбөгтэй хүү оройтой ба тэр нь 2-р орой юм. Иймд $c[5] = [2]$ болох ба уг процедур **traverse(2)** дуудалтыг хийнэ.
 - **traverse(2)**: 2-р оройн утга 30; түүнийг S рүү нэмснээр $S = [0, 10, 20, 30]$ болно. 2-р орой бөмбөгтэй хүүгүй тул энэ дуудалт дуусна.

- Программын биелэлт `traverse(5)` руу буцаж ирэх ба `s[5]` дахь бүх хүү оройг боловсруулж дууссан тул энэ дуудалт дуусна.
- **`traverse(4)`**: 4-р оройн утга 20; түүнийг S рүү нэмснээр $S = [0, 10, 20, 30, 20]$ болно. 4-р оройд бөмбөгтэй хүү байхгүй тул энэ дуудалт дуусна.

Одоо бүх дуудалт дууссан ба өөр боловсруулах орой үлдээгүй байна. Эцэст нь бүх бөмбөгийг машинаас гаргаж, `collect` процедур $S = [0, 10, 20, 30, 20]$ массивыг буцаана.

Таны даалгавар бол оройн тоо болох N тоог тодорхойлж, үндэс болон навчнаас бусад оройнууд өөр тэмдэглэгээтэй байх боломжтой үед машины бүтцийг илэрхийлэх эцгүүдийн $R = [R[0], R[1], \dots, R[N-2]]$ массивыг олох явдал юм. Формал байдлаар бол буцааж буй R массивыг зөв гэж үзэхийн тулд машины u орой бүрт дараах нөхцлүүд биелж байхаар ялгаатай $L[u]$ ($0 \leq L[u] < N$) тэмдэглээнүүдийг өгөх боломжтой байх юм:

- $L[u] = u$ ($0 \leq u < M$ ба $u = N-1$),
- $L[P[u]] = R[L[u]]$ ($0 \leq u < N-1$).

$R[u] > u$ байх **албагүй**. Таны шийдээ явуулах үед машин **хоосон байх ёстой**.

K нь `collect` үйлдлийг хийсэн тоо, B нь бүх `insert` дуудалтын үеийн бөмбөгний хамгийн их утга байг. $C = K + B$ гэж үзье. Тэгвэл C нь 1000-аас хэтэрч болохгүй. Зарим дэд бодлого дээр таны авах оноо C -гийн утгаас хамаарна.

Хэрэгжүүлэлтийн мэдээлэл

Та дараах процедурыг хэрэгжүүлнэ.

```
std::vector<int> find_structure(int M)
```

- M : машин дахь навчны тоо.
- Уг процедурыг тест бүрийн хувьд яг нэг удаа дуудна.
- Процедур эцгүүдийн зөв массив болох $N-1$ урттай $R = [R[0], R[1], \dots, R[N-2]]$ массивыг буцаана.

Машинтай харилцахын тулд таны процедур доорх хоёр процедурыг дуудаж болно.

Эхний процедур:

```
bool insert(int U, int X)
```

- U : бөмбөгний дээр нь бичигдэх ёстой навчны индекс. $0 \leq U < M$ нөхцөл биелэх ёстой.
- X : бөмбөгний бүхэл тоон утга. $0 \leq X \leq 1000$ гэсэн нөхцөл биелнэ.
- Хэрэв бөмбөгийг амжилттай оруулсан бол процедур `true` утга буцаах ба хэрэв U навч бөмбөгтэй байгаа бол `false` утгыг буцаана.
- Энэ процедурыг дээд тал нь 500 000 удаа дуудаж болно.

Хоёр дахь процедур:

```
std::vector<int> collect()
```

- Оруулсан бүх бөмбөгүүд дээрх утгуудыг цуглуулж, машиныг хоослоод, цуглуулсан массив S -г буцаана.

Хэрэв таны программын биелэлтийн үед C утга 1000-аас хэтэрвэл таны бодолт `Output isn't correct: Too many resources used` гэсэн алдааг авах болно.

Машины бүтэц нь `find_structure` процедурыг дуудахаас өмнө өөрчлөгдөхөө больсон байна. Шалгагч нь **детерминистик** байх ба өөрөөр хэлбэл та түүнийг хоёр удаа ажиллуулахдаа яг ижил үйлдлүүдийн дарааллыг гүйцэтгэсэн бол `collect` процедурын дуудалтууд нь ижил массивыг буцаана.

Хязгаарлалт

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Дэд бодлого

Дэд бодлого	Оноо	Нэмэлт хязгаарлалт
1	5	Үндэс нт яг M хүүхэдтэй.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Нэмэлт хязгаарлалт байхгүй.

Дэд бодлого 3 болон 4 дээр таны авах оноо доорх байдлаар C утгаас хамаарна.

Дэд бодлого 3 (25 оноо)

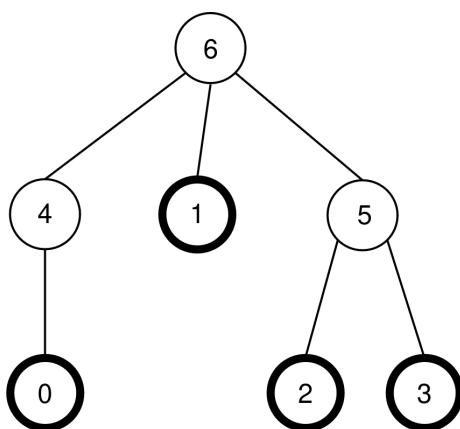
Хязгаарлалт	Оноо
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Дэд бодлого 4 (60 оноо)

Хязгаарлалт	Оноо
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Жишээ

Бодлогын өгүүлбэрт байгаатай ижил машин байгаа гэж үзье. Тэгвэл $N = 7$, $M = 4$ байх ба $P = [4, 6, 5, 5, 6, 6]$ байна. Уг машиныг доорх зураг дээр дахин дүрсэлсэн:



Шалгагч дараах дуудалтыг хийнэ:

```
find_structure(4)
```

Уг процедур дараах дуудалтуудын дарааллыг гүйцэтгэж болно:

1. **insert(0, 0):** 0-р навч хоосон тул 0 утгатай бөмбөгийг 0-р навчин дээр байрлуулна. $P[0] = 4$ дугаар орой хоосон тул уг бөмбөг 4-р орой руу шилжинэ. $P[4] = 6$ дугаартай орой хоосон тул бөмбөг 6-р орой руу шилжинэ. 6-р орой нь үндэс тул шилжилт зогсоно. Дуудалт `true` утга буцаана.
2. **insert(3, 20):** 3-р орой хоосон тул 20 утгатай бөмбөгийг 3-р навч дээр байрлуулна. $P[3] = 5$ дугаартай орой хоосон тул уг бөмбөг 5-р орой руу шилжинэ. $P[5] = 6$ дугаартай орой бөмбөгтэй тул шилжилт зогсоно. Дуудалт `true` утга буцаана.
3. **insert(1, 10):** 1-р навч хоосон тул 10 утгатай бөмбөгийг 1-р навчин дээр байрлуулна. $P[1] = 6$ бөмбөгтэй тул уг бөмбөг 1-р оройн дээр үлдэнэ. Дуудалт `true` утгыг буцаана.
4. **insert(2, 30):** 2-р навч хоосон тул 30 утгатай бөмбөгийг 2-р навчин дээр байрлуулна. $P[2] = 5$ нь бөмбөгтэй тул уг бөмбөг 2-р оройн дээр үлдэнэ. Дуудалт `true` утгыг буцаана.

5. `insert(1, 25)`: 1-р навч бөмбөгтэй тул бөмбөгийг 1-р навч дээр байрлуулахгүй. Дуудалт `false` утга буцаана.
6. `insert(0, 20)`: 0-р навч хоосон тул 20 утгатай бөмбөгийг 0-р навчин дээр байрлуулна. $P[0] = 4$ дугаартай орой хоосон тул уг бөмбөг 4-р орой руу шилжинэ. $P[4] = 6$ нь бөмбөгтэй тул шилжилт зогсоно. Дуудалт `true` утга буцаана.

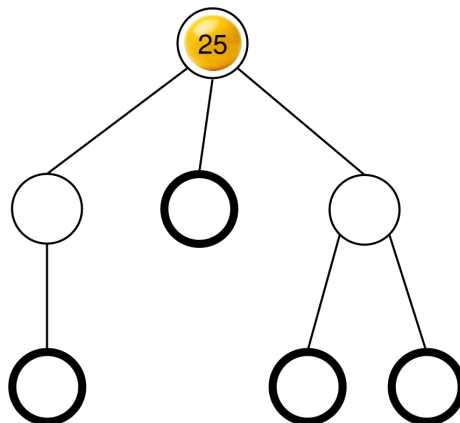
Үр дүнд нь гарах бөмбөгүүдийн байрлалыг бодлогын өгүүлбэрт үзүүлсэн байгаа.

Дараа нь уг процедур доорх дуудалтыг хийж болно:

```
collect()
```

Уг дуудалтын биелэлтийг бодлогын өгүүлбэрт тайлбарласан ба энэ дуудалт $S = [0, 10, 20, 30, 20]$ массивыг буцаана. Үүний дараа машин дахин хоосон болно.

Дараа нь процедур `insert(2, 25)` дуудалтыг хийж болно. 2-р навч хоосон тул 25 утгатай бөмбөгийг 0-р навч дээр байрлуулна. Уг бөмбөг 5-р орой руу шилжиж, дараа нь 6-р орой руу шилжээд тэндээ зогсоно. Энэ дуудалт `true` утга буцаана. Үр дүнд нь гарах бөмбөгүүдийн байрлалыг доорх зурагт үзүүлэв.



Эцэст нь процедур `collect()` дуудалтыг хийж болох ба энэ нь $S = [25]$ утгыг буцааж машиныг хоосолно.

`find_structure` процедурын буцааж чадах хоёр зөв эцгүүдийн массив байгаа: $R = P = [4, 6, 5, 5, 6, 6]$ ($L = [0, 1, 2, 3, 4, 5, 6]$ гэсэн тэмдэглэлтэнд харгалзах) ба $R = [5, 6, 4, 4, 6, 6]$ ($L = [0, 1, 2, 3, 5, 4, 6]$ тэмдэглэлтэнд харгалзах). Энэ жишээнд, $K = 2$ ба $B = 30$ тул $C = 32$ байна.

Жишээ шалгагч

Оролтын формат:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Гаралтын формат:

K B C

Q

R[0] R[1] R[2] ... R[Q-1]

Энд, Q нь `find_structure` процедурын буцаасан R массивын урт юм.